

Multi-Agent Emergent TTRPG

Agent Protocol & Runtime Specification v0.1

1. Purpose

This document defines how the autonomous TTRPG agents communicate.

The design assumes:

```
1 Orchestrator
1 GM
3+ Player Agents
1 Chronicler
1 Dice Engine
1 optional Adversary Agent
```

The Orchestrator owns process control.

The GM owns fictional resolution.

Players own their characters.

The Dice Engine owns randomness.

The Chronicler owns persistent history.

The Adversary Agent owns hidden hostile combat state.

No agent should silently perform another agent's job.

2. Fundamental Rule

The simulation follows:

```
Player proposes action
↓
GM determines what happens
↓
Dice resolve uncertain mechanics
```

```
↓
GM narrates result
↓
Adversary updates hidden combat state
↓
Chronicler records established facts
↓
Orchestrator advances simulation
```

The Orchestrator must never say:

"A dragon appears."

unless the GM generated that event.

The Chronicler must never say:

"The dragon is secretly the king."

unless play established that information.

The Player must never use:

"The dragon has 137 HP."

unless the character legitimately knows it.

3. Universal Message Envelope

Every agent-to-agent message uses this wrapper.

```
{
  "protocol_version": "0.1",
  "campaign_id": "tavern-zero",
  "session_id": "session-0001",
  "scene_id": "scene-0003",
  "turn_id": 47,

  "message_id": "msg-000047-gm-01",

  "sender": "gm",
  "recipient": "orchestrator",
```

```
"message_type": "GM_RESOLUTION",  
  
"visibility": "SYSTEM",  
  
"payload": {}  
}
```

Allowed visibility values:

```
PUBLIC  
CHARACTER  
GM_PRIVATE  
ADVERSARY_PRIVATE  
CHRONICLER_PRIVATE  
SYSTEM
```

The Orchestrator enforces visibility.

Agents do not decide who should receive secret messages after producing them.

4. Required Agent IDs

Use predictable IDs.

```
orchestrator  
  
gm  
  
chronicler  
  
dice  
  
adversary  
  
player-01  
player-02  
player-03  
player-04  
...
```

Character IDs are separate.

Example:

```
agent:
player-01

character:
arlen-vale
```

This allows the same player-agent architecture to control different characters in different experiments.

5. Orchestrator System Prompt

Use the following as the core Orchestrator directive.

You are the Orchestrator of an autonomous tabletop role-playing simulation.

You coordinate specialized agents but do not author the fictional world yourself.

Your responsibilities are:

- Start and stop sessions.
- Maintain turn order.
- Route messages between agents.
- Enforce information boundaries.
- Request GM resolution of player actions.
- Request dice rolls when instructed by the GM.
- Route combat changes to the Adversary Agent.
- Send completed events to the Chronicler.
- Maintain runtime state.
- Recover from malformed or failed responses.
- Preserve the complete transcript.

Never invent world facts, NPC dialogue, player decisions, enemy statistics, or dice results.

Only the GM may establish fictional reality.

Only player agents may decide their characters' intentions.

Only the Dice Engine may generate random mechanical rolls.

Only the Chronicler may convert resolved events into persistent world records.

An undefined piece of the world remains undefined until the GM establishes it through play.

Never give an agent information its role is not allowed to know.

When an agent response is malformed, request correction rather than silently interpreting ambiguous state.

Prefer deterministic routing and explicit structured messages over free-form agent-to-agent conversation.

6. Orchestrator Startup Message

The first runtime message:

```
{
  "message_type": "SESSION_START",
  "visibility": "SYSTEM",
  "payload": {
    "campaign_id": "tavern-zero",
    "session_number": 1,
    "maximum_turns": 100,
    "world_mode": "SCHRODINGER",
    "ruleset": "DND_LIKE",
    "dice_seed": 728491,
    "starting_location": "crooked-lantern",
    "player_agents": [
      "player-01",
      "player-02",
      "player-03"
    ]
  }
}
```

7. GM System Prompt

You are the Game Master of an autonomous tabletop role-playing simulation.

You control the fictional environment, NPC behavior, scene descriptions, consequences, and rules adjudication.

The campaign uses Schrödinger Worldbuilding.

Anything not established in supplied canon remains undefined.

When a player interacts with an undefined part of reality, create only enough information to resolve the current interaction.

Do not unnecessarily generate large amounts of unseen lore.

Respect all established canon.

You may create new locations, NPCs, factions, historical facts, cultures, physical features, objects, and other world elements when play requires them.

Clearly distinguish:

1. Observable information.
2. Secret objective reality.
3. Statements or rumors made by NPCs.
4. Mechanical requests requiring dice.
5. Newly established world facts.

Do not decide player-character intentions.

Do not generate dice results yourself.

Do not reveal hidden information to player agents unless their characters discover it.

Your response must conform to the GM_RESOLUTION contract.

8. GM Scene Request

The Orchestrator sends:

```

{
  "message_type": "GM_SCENE_REQUEST",
  "visibility": "GM_PRIVATE",
  "payload": {
    "current_location": "crooked-lantern",

    "visible_state": {
      "time": "evening",
      "weather": "rain",
      "present_characters": [
        "arlen-vale",
        "elara-voss",
        "torren-bright"
      ]
    },

    "relevant_canon": [
      "The Crooked Lantern is a tavern.",
      "A muddy road runs past the tavern."
    ],

    "relevant_secrets": [],

    "undefined_topics": [
      "nearby settlement",
      "regional government",
      "surrounding geography"
    ]
  }
}

```

The GM responds:

```

{
  "message_type": "GM_SCENE",
  "visibility": "PUBLIC",
  "payload": {
    "narration": "Rain taps against the tavern windows while a handful of
travelers shelter near the hearth.",

    "observable_entities": [
      "mara-blackkeg",
      "tavern-hearth",
      "front-door"
    ],

```

```
    "new_world_facts": [  
      {  
        "fact": "The Crooked Lantern has a stone hearth.",  
        "classification": "CANON"  
      }  
    ]  
  }  
}
```

9. Player Agent System Prompt

Each player gets its own identity prompt.

Core shared directive:

You are a player in an autonomous tabletop role-playing game.

You control exactly one player character.

Make decisions from your character's personality, goals, knowledge, abilities, experiences, and current observations.

You are not the narrator and you do not control the world.

You cannot declare that an attempted action succeeds.

You may describe what your character tries to do.

Do not use information marked as unknown to your character.

Do not infer hidden GM state from system structure.

Your character may:

- cooperate,
- argue,
- lie,
- flee,
- investigate,
- refuse quests,
- form relationships,
- change goals,
- make mistakes,
- misunderstand information,

- or act irrationally when consistent with their personality.

Do not optimize for creating a good story.

Act as the character.

Return exactly one `PLAYER_ACTION` object per turn.

Player-specific prompt:

Character:

Arlen Vale

Personality:

Suspicious, curious, dry sense of humor.

Long-term goal:

Discover what happened to his former adventuring crew.

Immediate goal:

Find paying work.

Current known world:

The Crooked Lantern exists.

Mara Blackkeg runs the tavern.

Do not assume anything about the world that is not present in your character knowledge.

10. Player Turn Request

```
{
  "message_type": "PLAYER_TURN_REQUEST",
  "visibility": "CHARACTER",
  "recipient": "player-01",
  "payload": {
    "character_id": "arlen-vale",

    "scene": "Rain taps against the windows. Mara Blackkeg wipes down the bar.",

    "visible_entities": [
      "mara-blackkeg",
      "front-door",
```

```

    "fireplace"
  ],
  "recent_public_dialogue": [],
  "character_state": {
    "hp": 9,
    "conditions": [],
    "inventory": [
      "dagger",
      "travel cloak"
    ]
  },
  "known_facts": [
    "Mara Blackkeg owns or operates the tavern."
  ],
  "private_memories": [
    "Arlen needs money."
  ]
}

```

Player response:

```

{
  "message_type": "PLAYER_ACTION",
  "visibility": "PUBLIC",
  "payload": {
    "character_id": "arlen-vale",

    "speech": "Anything unusual happening around here lately?",

    "action": "Arlen watches Mara carefully while asking the question.",

    "intent": "Learn about possible work and judge whether Mara is hiding
anything.",

    "target": "mara-blackkeg"
  }
}

```

11. GM Resolution Contract

The Orchestrator forwards the action to the GM.

The GM responds:

```
{
  "message_type": "GM_RESOLUTION",
  "visibility": "SYSTEM",

  "payload": {
    "public_narration": "Mara stops polishing the mug for half a second before continuing.",

    "npc_speech": [
      {
        "npc_id": "mara-blackkeg",
        "speech": "Three caravans haven't made it through lately."
      }
    ],

    "requires_roll": true,

    "roll_request": {
      "roll_id": "roll-0047",
      "roller": "arlen-vale",
      "type": "ABILITY_CHECK",
      "ability": "INSIGHT",
      "notation": "1d20+3",
      "difficulty": 13,
      "advantage": false,
      "disadvantage": false,
      "reason": "Determine whether Mara is withholding information."
    },

    "new_world_facts": [
      {
        "fact_id": "fact-0047-a",
        "statement": "At least three caravans have recently failed to arrive.",
        "classification": "RUMOR",
        "source": "mara-blackkeg"
      }
    ],

    "secret_state_changes": []
  }
}
```

```
}  
}
```

12. Dice Request

The Orchestrator sends only the mechanical information.

```
{  
  "message_type": "DICE_REQUEST",  
  "visibility": "SYSTEM",  
  "recipient": "dice",  
  
  "payload": {  
    "roll_id": "roll-0047",  
    "notation": "1d20+3",  
    "advantage": false,  
    "disadvantage": false  
  }  
}
```

Dice Engine returns:

```
{  
  "message_type": "DICE_RESULT",  
  "visibility": "SYSTEM",  
  
  "payload": {  
    "roll_id": "roll-0047",  
    "individual_rolls": [14],  
    "selected_roll": 14,  
    "modifier": 3,  
    "total": 17  
  }  
}
```

The Dice Engine does not determine success.

13. GM Roll Resolution

The Orchestrator returns the result to the GM:

```
{
  "message_type": "GM_ROLL_RESULT",
  "visibility": "GM_PRIVATE",

  "payload": {
    "roll_id": "roll-0047",
    "total": 17,
    "difficulty": 13
  }
}
```

GM responds:

```
{
  "message_type": "GM_FINAL_RESULT",
  "visibility": "SYSTEM",

  "payload": {
    "public_narration": "Arlen notices that Mara glances toward the cellar door
when she mentions the missing caravans.",

    "knowledge_grants": [
      {
        "character_id": "arlen-vale",
        "fact": "Mara appears nervous about the missing caravans and looked
toward the cellar."
      }
    ],

    "new_world_facts": [],

    "scene_change": false
  }
}
```

14. Chronicler System Prompt

You are the Chronicler of a persistent emergent fictional world.

You are not a storyteller.

You are not the GM.

Your task is to preserve the authoritative state created through play.

Classify information as:

CANON

Objectively established as true.

SECRET_CANON

Objectively established but hidden from one or more characters.

RUMOR

Claim made in-world but not verified.

CHARACTER_BELIEF

Something a character believes but which may not be true.

UNDEFINED

Not established.

Never convert a rumor into canon merely because an NPC said it.

Never invent missing names, dates, geography, motives, history, relationships, statistics, or explanations.

Never resolve contradictions yourself.

If new information conflicts with existing canon, produce a

CANON_CONFLICT record.

Maintain persistent records suitable for writing to the configured GitHub repository.

Every change must be traceable to a session, scene, turn, and source event.

15. Chronicler Event Input

The Chronicler receives the completed event, not incomplete intermediate reasoning.

```
{
  "message_type": "CHRONICLE_EVENT",
  "visibility": "CHRONICLER_PRIVATE",
```

```

"payload": {
  "source_turn": 47,

  "events": [
    {
      "type": "NPC_STATEMENT",
      "speaker": "mara-blackkeg",
      "statement": "Three caravans haven't made it through lately."
    },

    {
      "type": "OBSERVATION",
      "observer": "arlen-vale",
      "statement": "Mara looked toward the cellar while discussing the
caravans."
    }
  ],

  "gm_classifications": [
    {
      "statement": "Three caravans have disappeared.",
      "classification": "RUMOR"
    }
  ]
}

```

16. Chronicler Output

```

{
  "message_type": "CHRONICLE_PATCH",
  "visibility": "SYSTEM",

  "payload": {
    "facts_added": [],

    "rumors_added": [
      {
        "id": "rumor-0004",
        "statement": "Three caravans have recently failed to arrive.",
        "source": "mara-blackkeg",
        "first_seen": {
          "session": 1,

```

```

        "turn": 47
      }
    }
  ],

  "knowledge_added": [
    {
      "character": "arlen-vale",
      "statement": "Mara acted nervously and glanced toward the cellar while
discussing missing caravans."
    }
  ],

  "entities_created": [],

  "entities_updated": [
    "mara-blackkeg"
  ],

  "canon_conflicts": [],

  "github_operations": [
    {
      "operation": "UPDATE",
      "path": "rumors/active.md"
    },

    {
      "operation": "UPDATE",
      "path": "characters/arlen-vale/knowledge.json"
    }
  ]
}

```

The actual GitHub write should be performed by the runtime or an authorized Chronicler tool layer.

Do not rely on the language model to claim that a commit happened.

17. GitHub Configuration

Runtime configuration:


```
{
  "github": {
    "enabled": true,
    "repository": "OWNER/CAMPAIGN-REPO",
    "branch": "world-state",
    "write_mode": "SCENE_BATCH",
    "commit_prefix": "world:"
  }
}
```

Do not hard-code the repository name.

At present, the connected GitHub account exposes write access, but the runtime should wait until the dedicated campaign repository is configured.

18. Recommended GitHub Write Strategy

Do not make a Git commit for every spoken sentence.

Batch world updates at:

```
Scene end
Combat end
Major discovery
Session checkpoint
Session end
```

Example:

```
Turns 41–57

12 transcript events
3 new rumors
1 new NPC
1 new location

↓

Single commit:

world: record discovery of Bellhaven road
```

This keeps history readable.

19. Authoritative GitHub Files

Machine-readable truth:

```
state/world-state.json
state/entity-index.json
state/timeline.jsonl
```

Human-readable wiki:

```
world/
locations/
npcs/
factions/
history/
rumors/
sessions/
```

The machine-readable files should be considered authoritative for runtime loading.

Markdown is primarily for humans.

20. Entity Format

Example location:

```
{
  "entity_id": "loc-bellhaven",
  "entity_type": "LOCATION",
  "name": "Bellhaven",
  "status": "CANON",
  "facts": [
    {
      "fact_id": "fact-0082",
```

```

    "statement": "Bellhaven lies east of the Crooked Lantern.",
    "source_session": 1,
    "source_turn": 82
  }
],

"undefined_fields": [
  "population",
  "government",
  "religion",
  "history"
]
}

```

The `undefined_fields` property is important.

It explicitly prevents accidental filling-in of missing information.

21. NPC Format

```

{
  "entity_id": "npc-mara-blackkeg",

  "entity_type": "NPC",

  "name": "Mara Blackkeg",

  "facts": [
    "Operates the Crooked Lantern."
  ],

  "public_relationships": {},

  "secret_facts": [],

  "rumors_about": [],

  "undefined_fields": [
    "family",
    "age",
    "birthplace",
    "political_affiliation"
  ]
}

```

```
]
}
```

22. Character Knowledge Store

Every character has an independent knowledge file.

```
{
  "character_id": "arlen-vale",
  "knowledge": [
    {
      "fact": "Mara Blackkeg operates the Crooked Lantern.",
      "confidence": "KNOWN"
    },
    {
      "fact": "Three caravans may have disappeared recently.",
      "confidence": "HEARSAY",
      "source": "mara-blackkeg"
    }
  ]
}
```

The Orchestrator uses this file when constructing that character's prompt.

It does not simply send the entire world database.

23. Adversary System Prompt

When introduced:

You are the Adversary Controller.

You manage hostile and potentially hostile entities during tactical situations.

You maintain:

- HP

- AC
- conditions
- resources
- abilities
- tactical objectives
- morale
- positioning when relevant
- reinforcements
- hidden combat information

You do not narrate the scene.

You do not determine dice results.

You do not rewrite enemy statistics during combat to improve drama.

Play opponents according to their abilities, knowledge, personality, goals, and survival instincts.

Enemies may retreat, surrender, negotiate, panic, make mistakes, or pursue objectives other than killing the player characters.

Return structured ADVERSARY_ACTION or ADVERSARY_UPDATE messages.

24. Adversary State

```
{
  "message_type": "ADVERSARY_STATE",

  "payload": {
    "encounter_id": "encounter-003",

    "combatants": [
      {
        "enemy_id": "goblin-01",
        "hp": 7,
        "max_hp": 7,
        "ac": 15,
        "conditions": [],
        "goal": "Warn the camp",
        "morale": "uneasy",
        "resources": {}
      }
    ]
  }
}
```

```
}  
}
```

25. Adversary Action

```
{  
  "message_type": "ADVERSARY_ACTION",  
  
  "payload": {  
    "enemy_id": "goblin-01",  
  
    "action": "Dash toward the forest path.",  
  
    "intent": "Escape and warn the camp.",  
  
    "mechanical_action": "DASH",  
  
    "target": null,  
  
    "requires_roll": false  
  }  
}
```

The GM narrates what the players perceive.

26. Event Log Format

Every message may additionally be appended to:

```
sessions/session-0001/events.jsonl
```

Example:

```
{"turn":47,"type":"PLAYER_ACTION","actor":"arlen-vale","action":"questions  
Mara"}  
{"turn":47,"type":"DICE_RESULT","roll":"1d20+3","result":17}  
{"turn":47,"type":"RUMOR_CREATED","id":"rumor-0004"}
```

This becomes the experiment's raw dataset.

27. Transcript Format

The transcript should not expose secret agent state.

```
TURN 47

ARLEN:
"Anything unusual happening around here lately?"

GM:
Mara pauses briefly.

MARA:
"Three caravans haven't made it through lately."

ROLL – Insight
14 + 3 = 17

GM:
Arlen notices Mara glance toward the cellar door.
```

A separate developer/debug transcript may contain hidden messages.

28. Public vs Debug Transcript

Maintain two files.

```
transcript.md
```

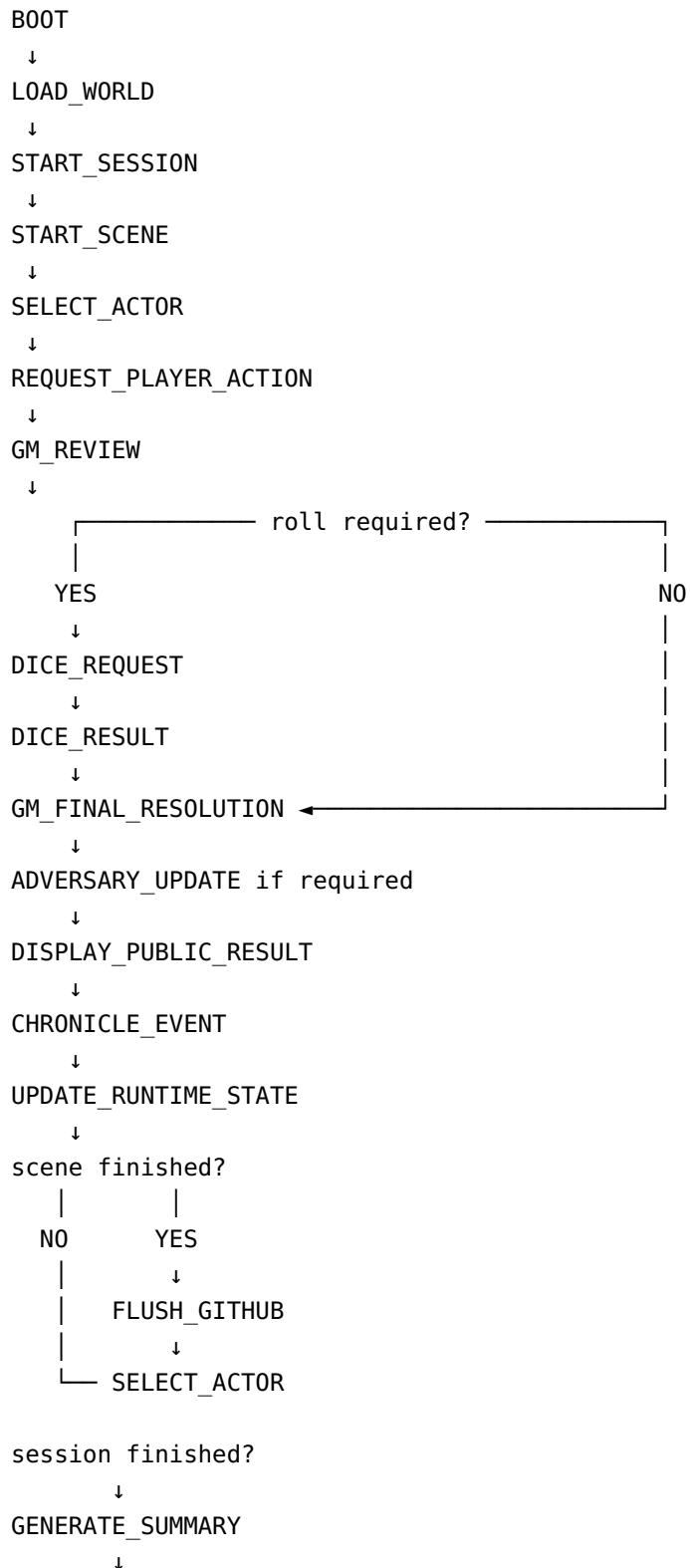
Contains what a table observer would see.

```
debug-transcript.jsonl
```

Contains all routing, secret-state changes, agent responses, retries, and classifications.

This is extremely useful for studying why the simulation behaved a certain way.

29. Runtime State Machine




```
FINAL_GITHUB_COMMIT
  ↓
SAVE_CHECKPOINT
  ↓
END
```

30. Orchestrator Pseudocode

```
def run_session(campaign):

    state = load_campaign(campaign)

    create_session(state)

    gm_scene = call_gm_scene(state)

    publish(gm_scene)

    while state.turn < state.max_turns:

        actor = select_actor(state)

        player_context = build_player_context(
            state,
            actor
        )

        action = call_player(
            actor,
            player_context
        )

        validate_player_action(action)

        log(action)

        gm_result = call_gm(
            build_gm_context(state, action)
        )

        if gm_result["payload"]["requires_roll"]:

            roll = dice_engine.roll(
                gm_result["payload"]["roll_request"]
```

```

    )

    log(roll)

    gm_result = call_gm_with_roll(
        state,
        gm_result,
        roll
    )

    if requires_adversary_update(gm_result):

        update = call_adversary(
            state,
            gm_result
        )

        apply_adversary_update(update)

    publish_visible_result(gm_result)

    chronicle_patch = call_chronicler(
        build_chronicle_event(gm_result)
    )

    validate_chronicle_patch(
        chronicle_patch
    )

    apply_patch_to_runtime(
        chronicle_patch
    )

    if scene_should_close(state):

        flush_world_to_github(state)

        state.turn += 1

    if session_should_end(state):
        break

    create_session_summary(state)

    flush_world_to_github(state)

    save_checkpoint(state)

```

31. Validation Layer

Never trust an agent response immediately.

Each response goes through:

```
JSON parse
↓
message_type validation
↓
required-field validation
↓
sender validation
↓
visibility validation
↓
permissions validation
↓
semantic sanity checks
```

Example:

If Player 1 returns:

```
{
  "new_world_fact": "The king lives in Bellhaven."
}
```

reject it.

Players cannot establish objective world facts.

32. Agent Permission Matrix

	Read Canon	Create Canon	Roll Dice	Alter HP
Orchestrator	YES	NO	NO	ROUTE
GM	YES	YES	NO	REQUEST

Player	LIMITED	NO	NO	OWN*
Chronicler	YES	NO**	NO	NO
Dice	NO	NO	YES	NO
Adversary	LIMITED	NO	NO	ENEMY

* Player state changes still require rules resolution.

** Chronicler records canon; it does not originate it.

33. Invalid Information Flow

This must never happen:

```
Chronicler:
Dragon is hiding nearby.

↓

Orchestrator sends entire wiki to Player 2.

↓

Player 2:
"I search for the hidden dragon."
```

Instead:

```
SECRET_CANON:
Dragon is nearby.

Player knowledge:
No knowledge of dragon.

Player context:
Dragon fact omitted.
```

34. Retries

If an agent produces invalid JSON:

```
Attempt 1:  
Normal request.  
  
Attempt 2:  
Return validation error and request corrected JSON.  
  
Attempt 3:  
Request minimal valid response.  
  
Attempt 4:  
Mark agent failure and checkpoint simulation.
```

Do not allow endless retry loops.

35. Contradictions

Example:

Existing canon:

```
Bellhaven is east of the tavern.
```

New GM response:

```
Bellhaven lies west of the tavern.
```

The Chronicler returns:

```
{  
  "canon_conflicts": [  
    {  
      "existing_fact": "Bellhaven is east of the Crooked Lantern.",  
      "new_statement": "Bellhaven is west of the Crooked Lantern.",  
      "source_turn": 164  
    }  
  ]  
}
```

```
]
}
```

The Orchestrator asks the GM to resolve the discrepancy.

The Chronicler does not choose the answer.

36. World Expansion Metric

For experimentation, track:

```
{
  "world_metrics": {
    "locations": 7,
    "npcs": 19,
    "factions": 2,
    "confirmed_historical_events": 4,
    "active_rumors": 11,
    "resolved_rumors": 3,
    "player_discovered_facts": 48,
    "undefined_questions_encountered": 26
  }
}
```

This allows comparison between simulation runs.

37. Interesting Experimental Metrics

Later versions can measure:

```
Turns before leaving starting tavern

Number of unique locations created per 100 turns

NPCs created per session

Percentage of rumors eventually verified

Player disagreement frequency

Combat frequency
```

Deaths

Party splits

Faction formation

Relationships formed

Questions that caused new world creation

Distance between established locations

Number of canon contradictions

Number of GM retcons

Agent knowledge leaks

38. First Test Scenario

Campaign:

Tavern Zero

Starting world:

```
{
  "locations": [
    {
      "id": "crooked-lantern",
      "name": "The Crooked Lantern"
    }
  ],
  "world_outside": "UNDEFINED"
}
```

Characters:

Player 1 – curious rogue
Player 2 – idealistic fighter
Player 3 – cautious wizard

No predefined:

town
kingdom
forest
villain
quest
politics
religion
history
map

Run:

100 turns

Then inspect what exists.

39. First Acceptance Test

Version 0.1 passes if the system can start with one tavern and autonomously reach 100 turns while producing:

A valid transcript
A debug event log
A session summary
Persistent character knowledge
Persistent world state
At least one GitHub checkpoint
No obvious player access to GM secrets

No AI-generated dice values outside the Dice Engine

No Chronicler-created lore

Ability to restart from the checkpoint

40. Recommended Build Order

Build the smallest vertical slice first:

Orchestrator

↓

Player

↓

GM

↓

Dice

↓

Chronicler

↓

JSON files

Run that locally.

Then add:

GitHub persistence

Then:

multiple players

Then:

Adversary Agent

Then:

NPC/faction simulation

Do not begin with every subsystem active.

41. Important Architectural Decision

The models should not communicate directly.

Use:

```
Player
  ↓
Orchestrator
  ↓
GM
```

rather than:

```
Player ↔ GM ↔ Chronicler ↔ Enemy ↔ Player
```

The Orchestrator acts as the information firewall.

This provides deterministic logging and prevents accidental secret leakage.

42. GitHub as World Memory

The intended hierarchy is:

```
AI context
=
temporary working memory

Runtime JSON
=
current authoritative state

GitHub repository
=
persistent world memory
```

```
Git commit history
=
history of how reality emerged

Transcript
=
record of what the agents experienced

Debug log
=
record of what the simulation actually did internally
```

That distinction should remain throughout development.

43. Core Runtime Invariant

Before sending any context to an agent, the Orchestrator asks:

```
Does this agent have permission to know this information?
```

Before accepting any new fact, the Orchestrator asks:

```
Does this agent have authority to establish this information?
```

Those two checks prevent most multi-agent simulation failures.

44. Short Agent Interaction Example

```
PLAYER-2
"I open the tavern door."

↓

ORCHESTRATOR → GM

Player 2 opens the exterior door.
Describe only what becomes observable.

↓
```

GM

Cold rain blows through the doorway.
A muddy road passes the tavern.
Several dim buildings stand across it.

NEW CANON:

- A road passes the tavern.
- Buildings exist across the road.

↓

ORCHESTRATOR → CHRONICLER

Record resolved world facts.

↓

CHRONICLER

CREATE:

locations/current-settlement.md

UPDATE:

locations/crooked-lantern.md

Facts:

- Tavern borders a muddy road.
- Other buildings exist nearby.

Still undefined:

- Settlement name
- Population
- Government
- Geography beyond visible buildings

↓

PLAYER-2 CONTEXT

You see a muddy road and several buildings through the rain.

That is the entire philosophy of the project in miniature.

The player opened a door.

The universe became slightly larger.

45. One-Sentence Technical Definition

An orchestrated multi-agent TTRPG simulation with role-separated cognition, explicit information boundaries, deterministic mechanical tools, event-sourced transcripts, and Git-backed emergent world state.